



مشروع مادة البرمجة المتوازية - الفصل الدراسي 2026

اسم المشروع: محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

(High-Performance E-Commerce Backend Engine)

فكرة المشروع: يهدف المشروع إلى بناء نظام متكامل لمنظومة تجارة إلكترونية تتعامل مع آلاف الطلبات المتزامنة. يركز المشروع على تطبيق المفاهيم المتعلقة بالمتطلبات غير الوظيفية مثل الأداء، حماية البيانات من التضارب، إدارة الموارد، وأتمتة العمليات الخلفية، أكثر من التركيز على كثرة الوظائف أو تنوع الصفحات. الهدف هو تطبيق المفاهيم التي تمت دراستها في الجلسات الثمانية، لضمان استقرار النظام تحت الضغط العالي.

المتطلبات الوظيفية: يُترك للطالب حرية تصميم الوظائف الأساسية لمنظومة التجارة الإلكترونية (تسجيل مستخدمين، عرض منتجات، إضافة إلى السلة، إتمام الطلب والدفع، إدارة المخزون)، على أن تكون هذه الوظائف كافية لتطبيق المتطلبات غير الوظيفية المذكورة أدناه.

المتطلبات غير الوظيفية:

1. حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity):

بناء آلية تسمح لعدة مستخدمين بتعديل نفس المورد (مثل كمية منتج في المخزون) دون حدوث تضارب، مع إثبات معالجة مشكلة الـ Race Condition.

2. إدارة الموارد الحاسوبية (Resource Management & Capacity Control):

التحكم في عدد العمليات المتوازية التي ينفذها النظام، بحيث لا يستهلك الموارد بشكل مفرط (انهيار) ولا يقللها بشكل يبطئ الاستجابة

3. المعالجة غير المتزامنة (Asynchronous Queues):

نقل المهام التي لا يحتاج المستخدم لانتظارها (مثل إصدار الفواتير أو إرسال الإشعارات) خارج المسار الرئيسي للطلب.

4. معالجة البيانات الضخمة على دفعات (Batch Processing): برمجة وظيفة خلفية (Background Job) تقوم بجدد المبيعات اليومية ومعالجتها كـ "دفعات" (Chunks) لزيادة سرعة الأداء.

5. توزيع الأحمال (Load Distribution): محاكاة توزيع الطلبات على أكثر من خادم مع توضيح وتبرير استراتيجية التوزيع المستخدمة.

6. استراتيجية التخزين المؤقت (Distributed Caching): دمج طبقة تخزين مؤقت (مثل Redis) لتخزين المنتجات الأكثر طلباً وتقليل الاستعلامات المباشرة من قاعدة البيانات.

7. التحكم في الأقفال (Concurrency Control): تطبيق مفهوم القفل المتفائل (Optimistic Locking) أو التشاؤمي (Pessimistic) عند تحديث كميات المخزون الحساسة.

8. سلامة المعاملات (Transaction Integrity / ACID): ضمان أن العمليات المركبة (دفع + تحديث مخزون + إنشاء طلب) تنجح كلها أو تفشل كلها، حتى تحت الوصول المتزامن.

9. اختبار الاستقرار تحت الضغط (Stress Testing): تقديم تقرير يثبت قدرة النظام على تخدم 100 مستخدم متزامن على الأقل دون انهيار أو فقدان بيانات.

10. القياس وتحديد الاختناقات (Benchmarking & Bottleneck Analysis): قياس زمن الاستجابة لأهم العمليات، تحديد اختناق واحد على الأقل، وتقديم مقارنة رقمية قبل التحسين وبعده.

ملاحظات:

- تعليمات تسليم المشروع:
- هناك مقابلات مرحلية إلزامية لمتابعة وتقييم التقدم، حتى يتم تسليم المشروع كاملاً:
- يتعين على كل طالب تفعيل الحس الهندسي النقدي عند معالجة أي مطلب؛ بحيث يكون قادراً على تقديم تفسير منهجي لكل حل يختاره، وكل أداة يعتمد عليها، وكل تقريب حسابي يلجأ إليه
- a. التوثيق: يجب إرفاق ملف نصي يشرح التصميم البرمجي (Architecture) وكيف تم تطبيق مفهوم البرمجة الموجهة للجوانب (AOP) لمراقبة الأداء.

- b. التعليمات البرمجية: كود مصدري نظيف مع تعليقات توضح نقاط المزامنة (Synchronization points) المستخدمة لضمان سلامة التزامن بين الخيوط (thread-safe concurrency).
- c. العرض التجريبي: جلسة مناقشة (ستكون على شكل جلسات تقييمية لعدة مرات) يتم فيها محاكاة ضغط عالي على النظام ومراقبة سلوك الخيوط وقاعدة البيانات.
- التقنيات المتاحة : Django , .NET , Spring Boot , Laravel

بالتوفيق